

Proyecto 1 — Registro de perritos de la calle

Materia: Programación lógica y funcional · Profesor: Daniel Varela · Última modificación:

2026-09-22 18:00

De qué se trata

En equipos de 3, van a construir una aplicación para registrar perritos de la calle: quien encuentra uno le toma una foto, le pone un nombre, anota cómo es y marca en un mapa dónde lo vio.

El registro sirve para que rescatistas, vecinos y asociaciones sepan qué perros hay, cómo identificarlos y en qué zona andan. Es un problema real, con usuarios reales, y así hay que tratarlo: la aplicación debe poder usarse desde un celular en la calle, no sólo desde la computadora del laboratorio.

El proyecto se trabaja como en un equipo de desarrollo de verdad: cada integrante tiene un rol, todo pasa por Git, y cualquier persona ajena al equipo debe poder instalar y correr el sistema siguiendo el README.

El equipo

Cada equipo tiene exactamente 3 integrantes y cada uno asume un rol. El rol dice de qué es responsable cada quien, no que los demás no puedan tocar esa parte.

Rol	Es responsable de	Entrega visible en el repositorio
Frontend	Pantallas, formulario, captura o carga de la foto, mapa con el pin, validaciones del lado del cliente, que se use bien en celular	Capturas de pantalla en el README
Backend	API que recibe y entrega los registros, validación del lado del servidor, almacenamiento de las imágenes, manejo de errores	Lista de endpoints en el README
DBA	Modelo de datos, script de creación de la base, catálogos (razas, colores), datos de prueba, respaldo	Scripts o migraciones de la base, diagrama entidad-relación

Reglas del equipo:

- Cada integrante debe tener commits propios en su parte. El historial de Git es la evidencia de quién hizo qué.
- En la demostración, cada quien responde únicamente por su rol: al frontend se le pregunta del frontend, al backend del backend y al DBA del modelo de datos. Nadie contesta por otro.
- Si un integrante deja de trabajar, el equipo lo reporta al profesor antes de la entrega, no el día de la entrega.

Qué debe hacer la aplicación

Un registro de perrito tiene estos campos. Lo obligatorio se valida en el frontend **y** en el backend: si alguien se salta el formulario y llama la API directo, el backend debe rechazar el registro incompleto.

Campo	¿Obligatorio?	Regla
Foto	Sí	Se puede tomar con la cámara del dispositivo o subir una imagen existente. Las dos opciones deben funcionar. Formatos JPG, PNG o WEBP
Nombre	Sí	Texto no vacío. Un nombre de puros espacios no cuenta

Campo	¿Obligatorio?	Regla
Raza	No	Se elige de un catálogo. Debe existir la opción "Sin raza definida / criollo"
Color principal	Sí	Exactamente uno, elegido de un catálogo
Colores adicionales	No	De 0 a 2, del mismo catálogo: máximo 3 colores en total contando el principal. No se puede repetir el color principal ni repetir un color
Ubicación	Sí	Un pin en un mapa. El usuario puede usar su ubicación actual o mover el pin a mano. Se guardan latitud y longitud
Fecha de registro	Automático	La pone el sistema, no el usuario

Además del formulario, la aplicación debe:

1. **Mostrar los perritos registrados en un mapa**, con un pin por perrito. Al tocar el pin se ve su foto, nombre y colores.
2. **Mostrar una lista** de los registros con foto en miniatura.
3. **Ver el detalle** de un registro.
4. **Mostrar errores entendibles**: "Falta la foto" sí; "Error 400" no.

No se pide inicio de sesión ni usuarios. Si un equipo lo quiere agregar, es un extra, no un requisito.

Ojo con la cámara: los navegadores sólo dan acceso a la cámara y a la ubicación en `https` o en `localhost`. Pruébenlo desde el celular antes de la entrega, no la noche anterior.

Los paradigmas en juego

Esta no es una aplicación web cualquiera: es donde se aplica lo que vimos en la unidad 1. Tres cosas se revisan explícitamente.

Declarativo e imperativo, conviviendo

- Las consultas de datos se resuelven en **SQL**, que es declarativo: filtrar, ordenar y agrupar le toca al manejador. Traer todos los registros y filtrarlos con un ciclo en el lenguaje de la aplicación es justo lo que no se debe hacer.
- Al menos una consulta con `JOIN` entre el perrito y sus colores, y una con agregación (por ejemplo, cuántos perritos hay por color o por zona).
- El README señala qué parte del sistema es declarativa y cuál imperativa, y por qué. Recuerden que la interfaz también declara: HTML y CSS describen qué debe verse, no cómo dibujarlo.

Idempotencia del registro

El formulario tiene exactamente el problema del ejercicio 3: el usuario presiona Enviar dos veces, o el celular reintenta por mala señal, y quedan dos perritos idénticos.

- El guardado debe ser **idempotente**: el segundo envío devuelve la misma respuesta y el mismo identificador, y no crea un registro nuevo. Responder "error: duplicado" no es idempotencia.
- Usen una clave de idempotencia generada al abrir el formulario, o una clave natural que puedan justificar. Documenten cuál eligieron y por qué.
- Tiene que haber una prueba del doble envío, y se demuestra en vivo.

Más de un paradigma

- En el README, una sección que diga qué paradigma se usó en cada parte y dónde está ese código: imperativo, orientado a objetos, funcional, declarativo.
- Al menos una transformación de datos resuelta en **estilo funcional**: con `map`, `filter` o `reduce`, sin mutar la estructura original y sin ciclos explícitos. Señalen cuál es.
- No se trata de usar cuatro lenguajes. Se trata de que reconozcan el paradigma en el que están escribiendo en cada momento y sepan explicar por qué ahí conviene ése.

Punto extra — acceso desde internet

Una aplicación que sólo corre en la laptop de quien la programó no le sirve a nadie en la calle. Esto es **opcional**: es el punto extra del proyecto y no afecta a quien no lo intente. Se divide en dos mitades que valen lo mismo: **explicar cómo se publica** y **lograr que se pueda entrar**. Hacer una sola de las dos suma la mitad.

Mitad 1 — Explicarlo

Una sección **Despliegue** en el README que conteste, hayan llegado o no a publicarlo:

- Dónde correría cada pieza: la aplicación, la base de datos y el directorio de las imágenes, que sigue sin poder vivir junto al código.
- Cómo se obtiene el dominio y el certificado para **HTTPS**. No es opcional: sin HTTPS el navegador no da cámara ni ubicación.
- Qué cambia entre la configuración local y la de producción, variable por variable, y dónde se guardan las contraseñas del servidor.
- Qué puertos quedan abiertos hacia internet y cuáles no. La base de datos no se expone.
- Cómo se respalda la base y las imágenes, y cómo se restauran.
- Como no hay Docker: la instalación es directa en el servidor, el servicio queda administrado por systemd o equivalente para que reviva solo, y al frente va un proxy inverso como nginx o Caddy.

Mitad 2 — Lograrlo

La aplicación accesible desde una **URL pública** el día de la demostración, probada desde un celular con datos móviles, no desde la red del laboratorio.

- Vale cualquier opción: un servicio de hospedaje gratuito, una máquina virtual, o un túnel tipo Cloudflare Tunnel, ngrok o Tailscale Funnel, siempre que esté documentado en el README junto con la URL.
- Si usan un túnel desde una laptop, tengan en cuenta que la liga cambia o se cae al cerrar la sesión. Prueben la URL desde otro dispositivo antes de presentar.
- El servicio de hospedaje no puede exigir Docker para desplegar.

Reglas técnicas

Tecnologías

La elección de lenguajes, frameworks y manejador de base de datos es libre. Lo que no es libre: tiene que correr **sin Docker** y todo debe quedar documentado en el README con sus versiones exactas.

Sin Docker significa: ningún `Dockerfile`, ningún `docker-compose.yml`, ninguna instrucción del tipo "instale Docker". El sistema se instala directamente en la máquina, como lo haría alguien que recibe el proyecto sin contenedores.

Almacenamiento de las imágenes

Las fotos de los perritos **no** pueden guardarse en ninguna carpeta que alcance los binarios o el código de la aplicación: ni dentro del código fuente, ni en `static/`, `public/`, `resources/`, `priv/static/` o la carpeta de despliegue del servidor.

- La ruta de almacenamiento se define en la configuración (por ejemplo `RUTA_IMAGENES`), apunta a un directorio fuera del proyecto y el README explica cómo crearlo.
- Las imágenes se sirven a través de un endpoint del backend, no exponiendo la carpeta directamente.
- El backend genera el nombre del archivo; nunca usa el nombre que mandó el usuario.
- Se valida que el archivo realmente sea una imagen, no sólo que su extensión lo diga.

La razón: un archivo subido por un usuario que queda junto al código puede terminar ejecutándose o sobrescribiendo algo de la aplicación. Separar los datos subidos de los binarios es una práctica básica de seguridad.

Git

- Un solo repositorio por equipo, público o compartido con el profesor desde el primer día.
- Nada se entrega por correo ni en USB: lo que no está en el repositorio no existe.
- Trabajo en ramas y fusión por *pull request*. Al menos un *pull request* revisado por otro integrante por cada rol.
- Mensajes de commit que digan qué cambió. "cambios", "ya quedó" y "asdf" no son mensajes.
- No se suben contraseñas, llaves de API, `node_modules/`, `target/`, entornos virtuales ni las fotos de prueba pesadas. Usen `.gitignore` y un archivo de configuración de ejemplo (`.env.example` o equivalente).

Inteligencia artificial

El uso de IA es **opcional** y no suma ni resta puntos. La única condición es que puedan explicar línea por línea cualquier código que entreguen, venga de donde venga: en la demostración se elegirá un fragmento al azar y quien lo escribió tendrá que explicarlo.

README

El `README.md` en la raíz del repositorio es parte de la calificación. La prueba es simple: una persona que nunca ha visto el proyecto, con una computadora limpia, debe poder instalarlo y ejecutarlo siguiéndolo al pie de la letra. Debe incluir:

1. Nombre del proyecto, integrantes y rol de cada uno.
2. Requisitos previos con versiones exactas (lenguaje, runtime, manejador de base de datos, etc.).
3. Pasos de instalación en orden, con los comandos listos para copiar.
4. Creación de la base de datos y carga de catálogos y datos de prueba.
5. Configuración: qué variables hay que definir y un ejemplo de cada una.
6. Cómo ejecutar el backend y el frontend, y en qué URL queda cada uno.
7. Cómo probarlo desde un celular en la misma red.
8. Lista de endpoints de la API.
9. Capturas de pantalla.
10. Problemas comunes y cómo resolverlos.
11. Sección **Paradigmas**: qué paradigma se usó en cada parte del sistema, dónde está ese código, y cómo se resolvió la idempotencia del registro.
12. *(Sólo si van por el punto extra)* Sección **Despliegue**: cómo se publica en internet, y la URL pública si ya está en línea.

Qué se entrega

Fecha de entrega y demostración: **por confirmar**. Se entrega la liga al repositorio; se califica el último commit de la rama `main` antes de la fecha límite.

La organización de carpetas del repositorio es libre y depende de las tecnologías que elijan. El README debe explicar dónde está cada parte; es el único archivo con ubicación fija, en la raíz del repositorio.

- ☐ Aplicación funcionando con todos los requisitos de la sección anterior.

- ☐ Base de datos con catálogo de al menos 10 razas y 10 colores, y al menos 15 perritos de prueba con foto.
- ☐ README completo con los 11 puntos obligatorios, incluida la sección de paradigmas.
- ☐ Prueba del doble envío que demuestre que el guardado es idempotente.
- ☐ Demostración en vivo de 10 minutos: registrar un perrito desde un celular (con foto tomada en ese momento), verlo aparecer en el mapa, y mostrar que la validación rechaza un registro incompleto.
- ☐ Instalación en vivo: el profesor elige una máquina del laboratorio y el equipo sigue su propio README frente al grupo. Si el README falla, se corrige ahí mismo y se anota como observación.
- ☐ (*Punto extra*) URL pública funcionando, abierta desde un celular con datos móviles frente al grupo, y la sección de despliegue en el README.

Cómo se califica

Escala 0–100. La calificación es de equipo, con un ajuste individual por la evidencia en Git y las respuestas en la demostración.

Criterio	Excelente (90–100)	Suficiente (70–79)	Insuficiente (< 70)	Peso
Trabajo en equipo con Git	Ramas, pull requests revisados, commits claros y repartidos entre los 3 integrantes a lo largo del proyecto	Commits de todos, pero todo en <code>main</code> o mensajes vagos	Un solo integrante sube todo, o todos los commits el último día	20 %
Declarativo e imperativo	El filtrado, ordenamiento y agregación ocurren en SQL; el README explica bien dónde está cada estilo y por qué	Usa SQL pero resuelve en código algo que tocaba a la base	Trae todo y filtra con ciclos; no distingue un estilo del otro	15 %

Criterio	Excelente (90–100)	Suficiente (70–79)	Insuficiente (< 70)	Peso
Uso de distintos paradigmas	Identifica los paradigmas en su propio código con ejemplos concretos; hay una transformación funcional sin mutación ni ciclos	Identifica los paradigmas, ejemplos genéricos	Copia definiciones o no reconoce lo que escribió	15 %
Idempotencia	El doble envío no duplica, devuelve el mismo id y conserva el registro original; hay prueba que lo demuestra	Evita el duplicado pero cambia la respuesta o el registro	El segundo envío crea otro perrito o responde con error de duplicado	15 %
Funcionalidad	Todos los campos y reglas funcionan; cámara y carga de archivo, mapa y lista operan desde celular	Registra, pero falla alguna regla (colores, cámara, pin)	No registra o pierde datos	20 %
README y ejecución sin Docker	Otra persona lo instala y corre siguiéndolo sin ayuda	Se instala con ayuda del equipo	No se logra ejecutar	15 %

Punto extra — acceso desde internet: suma hasta 10 puntos sobre la calificación del proyecto, sin pasar de 100. La mitad es por la explicación del despliegue y la otra mitad por tenerlo realmente en línea. Un equipo puede sacar 100 sin intentarlo.

Ajuste individual: un integrante sin commits propios en su rol, o que no pueda explicar el código de su propia parte, puede recibir una calificación menor que la de su equipo.

Preguntas frecuentes

¿Puede ser app móvil nativa en vez de web? Sí, siempre que se pueda instalar y ejecutar sin Docker siguiendo el README, y que la demostración se haga en un teléfono real.

¿Podemos usar un servicio en la nube para la base de datos o las imágenes? Sí, pero el README debe explicar cómo crear la cuenta y configurarla, y el sistema debe funcionar también con una base local para la instalación en vivo.

¿Qué mapa usamos? El que quieran. Si eligen uno que pide llave de API, la llave no se sube al repositorio y el README explica cómo obtenerla.

¿Guardamos la foto en la base de datos o en disco? Cualquiera de las dos vale si lo justifican en el README. Si es en disco, se aplica la regla de *Almacenamiento de las imágenes*: fuera de cualquier carpeta de la aplicación.

¿Se puede editar o borrar un registro? No es obligatorio. Si lo agregan, cuenta como extra.

¿Tenemos que usar IA? No. Es opcional y no cambia la calificación. Lo que sí se revisa es que entiendan lo que entregaron, sin importar de dónde salió.